



Faculty of engineering, Cairo University
Electronics and Electrical Communication Department
ELC4030 – Advanced Embedded Systems

Programming project

Name:	Roa Atef Mohamed
Sec:	2
ID:	9220291

Submitted to : Dr. Hany M. Elsayed

Table Of Contents

I. Introduction:	3
II. Code explanation:	3
1. timespec_add_us(struct timespec *t, long us)	3
2. timespec_cmp(struct timespec *a, struct timespec *b)	3
3. get_elapsed_ms()	4
4. get_time_us()	4
5. simulate_workload()	5
6. task_function(void *arg)	5
7. Main()	6
III. Results:	9
1. Rate monotonic (RM) scheduler (least period -highest priority)	9
1.1. Tasks information	9
1.2. Deadline satisfaction	9
1.3. Comment	10
2. Non RM scheduling	10
2.1. Comment	11
3. Single-core CPU utilization of the tasks	11
3.1. Comment	11
4. Single-core of the tasks without background applications	12
4.1. Deadline misses	12
4.2. CPU utilization	12
4.3. Comment	12
5. Single-core of the tasks with background applications	13
5.1. Deadline misses	13
5.2. CPU utilization	13
5.3. Comment	14
6. Comparison between parallel loads and only tasks running on single CPU	14
7. Multi-core of the tasks without background applications	15
7.1. Deadline misses	15
7.2. CPU utilization	15
7.3. Comment	15
8. Multi-core of the tasks with background applications	16
8.1. Deadline misses	16
8.2. CPU utilization	16
8.3. Comment	16
9. Comparison between parallel loads and only tasks running on multi core	17
IV. Full code	18

I. Introduction:

Rate-Monotonic Scheduling (RMS) is a fundamental fixed-priority scheduling algorithm used in real-time systems. It assigns priorities to periodic tasks based solely on their periods: the shorter the period, the higher the priority. This makes RMS both simple and powerful, as it guarantees optimal schedulability among all fixed-priority policies. In this assignment, RMS is applied to three periodic tasks with different periods, and their execution is measured to evaluate timing behavior, deadline satisfaction, and the impact of Linux's real-time scheduling mechanisms. Through this experiment, we demonstrate why RMS provides the best fixed-priority assignment and how task timing is affected by workload and system activity.

II. Code explanation:

1. `timespec_add_us(struct timespec *t, long us)`

Purpose: Adds a specified number of **microseconds** to a `timespec` structure.

Why: Used to calculate the **next activation time** of a periodic task in absolute terms.

How it works: -Adds the full seconds (`us / 1,000,000`) to `t->tv_sec`.

-Converts the remaining microseconds (`us % 1,000,000`) to nanoseconds and adds to `t->tv_nsec`.

-If `t->tv_nsec` exceeds 1,000,000,000, it wraps around into `t->tv_sec`.

```
// Add microseconds to a timespec structure
void timespec_add_us(struct timespec *t, long us) {
    t->tv_sec += us / 1000000;
    t->tv_nsec += (us % 1000000) * 1000;
    if (t->tv_nsec >= 1000000000) {
        t->tv_sec++;
        t->tv_nsec -= 1000000000;
    }
}
```

2. `timespec_cmp(struct timespec *a, struct timespec *b)`

Purpose: Compares two `timespec` values.

Why: Helps determine if a task **missed its deadline**.

Return values: 1 → a is later than b, -1 → a is earlier than b, 0 → a and b are equal.

How: First compares seconds, then nanoseconds if seconds are equal.

```
// Compare two timespec values
int timespec_cmp(struct timespec *a, struct timespec *b) {
    if (a->tv_sec != b->tv_sec)
        return (a->tv_sec > b->tv_sec) ? 1 : -1;
    if (a->tv_nsec != b->tv_nsec)
        return (a->tv_nsec > b->tv_nsec) ? 1 : -1;
    return 0;
}
```

3. get_elapsed_ms()

Purpose: Returns the **milliseconds elapsed** since the program started.

Why: Used to print task start and finish timestamps relative to program start.

How: -Reads current time using clock_gettime(CLOCK_MONOTONIC).

-Subtracts program_start_time and converts to milliseconds.

```
// Milliseconds since program start (wall clock time)
long get_elapsed_ms() {
    struct timespec current_time;
    clock_gettime(CLOCK_MONOTONIC, &current_time);

    long sec = current_time.tv_sec - program_start_time.tv_sec;
    long nsec = current_time.tv_nsec - program_start_time.tv_nsec;

    if (nsec < 0) {
        sec--;
        nsec += 1000000000;
    }
    return sec * 1000 + nsec / 1000000;
}
```

4. get_time_us()

Purpose: Returns the **current time in microseconds**.

Why: Measures the **execution time** of a task workload accurately.

How: -Reads current time using clock_gettime(CLOCK_MONOTONIC).

-Converts seconds to microseconds and adds nanoseconds divided by 1000.

```
// Current time in microseconds
long get_time_us() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000000 + ts.tv_nsec / 1000;
}
```

5. simulate_workload()

Purpose: Simulates CPU-bound work for a task.

Why: Represents actual computation for **real-time task testing**.

How: -Runs nested loops ($n_1 \times n_2$) performing dummy calculations.

-volatile prevents the compiler from optimizing away the loops.

```
// Simple nested loops used to generate CPU load
void simulate_workload() {
    volatile int a;
    for (long i = 0; i < n1; i++) {
        for (long j = 0; j < n2; j++) {
            a = j / 2;
        }
    }
    (void)a;
}
```

6. task_function(void *arg)

Purpose: Function executed by each thread representing a **periodic task**.

Steps per cycle:

1. Increment the **task's cycle counter**.
2. Print **start timestamp** using `get_elapsed_ms()`.
3. Execute `simulate_workload()` and measure execution time in microseconds.
4. Print **finish timestamp** and **execution duration**.
5. Compute **next activation time** based on the task period.
6. **Check for deadline miss**: if current time > next activation, print warning and increment counters.
7. **Sleep until next period** using `clock_nanosleep()` with `TIMER_ABSTIME` for precise periodicity.

Why: Implements **periodic real-time behavior** with deadline monitoring.

```
// Periodic task executed by each thread
void *task_function(void *arg) {
    int task_id = *(int *)arg;
    struct timespec next_wakeup = program_start_time;
    printf("Task %d thread started successfully\n", task_id);
    for (int k = 0; k < 500; k++) {
        // Update cycle count
        if (task_id == 1) task1_cycles++;
        if (task_id == 2) task2_cycles++;
        if (task_id == 3) task3_cycles++;
        // Print start timestamp (wall clock)
        long start_wall_ms = get_elapsed_ms();
        printf("Task %d started at %ld ms (Cycle %d)\n",
            task_id, start_wall_ms, k + 1);
        // Measure ACTUAL CPU execution time (excludes preemption time)
        long cpu_start_us = get_thread_cpu_time_us();
```

```

    long wall_start_us = get_time_us();

    simulate_workload();

    long cpu_end_us = get_thread_cpu_time_us();
    long wall_end_us = get_time_us();

    long cpu_time_us = cpu_end_us - cpu_start_us;
    long wall_time_us = wall_end_us - wall_start_us;

    // Print end timestamp (wall clock)
    long finish_wall_ms = get_elapsed_ms();
    printf("Task %d finished at %ld ms (Cycle %d)\n",
           task_id, finish_wall_ms, k + 1);

    // Execution duration in ms (actual CPU time only)
    printf("  Execution time: %.0f ms\n", cpu_time_us / 1000.0);

    // Compute next activation time
    next_wakeup.tv_sec += periods_ms[task_id - 1] / 1000;
    next_wakeup.tv_nsec += (periods_ms[task_id - 1] % 1000) * 1000000;
    if (next_wakeup.tv_nsec >= 1000000000) {
        next_wakeup.tv_sec++;
        next_wakeup.tv_nsec -= 1000000000;
    }

    // Check for deadline miss
    struct timespec now;
    clock_gettime(CLOCK_MONOTONIC, &now);

    if (timespec_cmp(&now, &next_wakeup) > 0) {
        printf("DEADLINE MISS: Task %d (Cycle %d)\n", task_id, k + 1);
        deadline_miss_detected = 1;

        if (task_id == 1) deadline_misses_task1++;
        if (task_id == 2) deadline_misses_task2++;
        if (task_id == 3) deadline_misses_task3++;
    }

    // Sleep until next period
    clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next_wakeup, NULL);
}

printf("Task %d completed all cycles\n", task_id);
return NULL;
}

```

7. Main()

Purpose: Sets up and starts the simulation of RMS tasks.

Steps:

1. Initialize **thread attributes** for real-time scheduling (policy `SCHED_FIFO`) and explicit priority.
2. Set **CPU affinity** so all tasks run on the same core.
3. Assign **RMS priorities** (shorter period → higher priority).

4. Record **program start time**.
5. Create **three threads**, each running `task_function` with its task ID.
6. Wait for threads to complete using `pthread_join()`.
7. Destroy thread attributes.
8. Print **execution summary**, including cycles completed and deadline misses.

```
int main() {
    printf("Rate Monotonic Scheduling\n");
    printf("=====\n");

    int task1_id = 1, task2_id = 2, task3_id = 3;
    pthread_t thread1, thread2, thread3;
    pthread_attr_t attr1, attr2, attr3;
    cpu_set_t cpu1, cpu2, cpu3;
    struct sched_param myparam;
    int ret;

    printf("Task Setup:\n");
    printf("Task 1 -> Period:100ms Priority:%d\n", rms_priorities[0]);
    printf("Task 2 -> Period:200ms Priority:%d\n", rms_priorities[1]);
    printf("Task 3 -> Period:300ms Priority:%d\n\n", rms_priorities[2]);
    printf("Workload: n1=%ld n2=%ld\n\n", n1, n2);

    // ----- Task 1 (highest priority) -----
    pthread_attr_init(&attr1);
    pthread_attr_setinheritsched(&attr1, PTHREAD_EXPLICIT_SCHED);

    ret = pthread_attr_setschedpolicy(&attr1, SCHED_FIFO);
    if (ret != 0) {
        printf("Warning: Cannot set SCHED_FIFO (need root). Using default scheduling.\n");
        pthread_attr_setschedpolicy(&attr1, SCHED_OTHER);
    } else {
        using_realtime = 1;
    }

    CPU_ZERO(&cpu1);
    CPU_SET(0, &cpu1);
    pthread_attr_setaffinity_np(&attr1, sizeof(cpu_set_t), &cpu1);

    if (using_realtime) {
        myparam.sched_priority = rms_priorities[0];
        pthread_attr_setschedparam(&attr1, &myparam);
    }

    // ----- Task 2 -----
    pthread_attr_init(&attr2);
    pthread_attr_setinheritsched(&attr2, PTHREAD_EXPLICIT_SCHED);
    pthread_attr_setschedpolicy(&attr2, using_realtime ? SCHED_FIFO : SCHED_OTHER);

    CPU_ZERO(&cpu2);
    CPU_SET(0, &cpu2);
    pthread_attr_setaffinity_np(&attr2, sizeof(cpu_set_t), &cpu2);

    if (using_realtime) {
        myparam.sched_priority = rms_priorities[1];
        pthread_attr_setschedparam(&attr2, &myparam);
    }
}
```

```

// ----- Task 3 -----
pthread_attr_init(&attr3);
pthread_attr_setinheritsched(&attr3, PTHREAD_EXPLICIT_SCHED);
pthread_attr_setschedpolicy(&attr3, using_realtime ? SCHED_FIFO : SCHED_OTHER);

CPU_ZERO(&cpu3);
CPU_SET(0, &cpu3);
pthread_attr_setaffinity_np(&attr3, sizeof(cpu_set_t), &cpu3);

if (using_realtime) {
    myparam.sched_priority = rms_priorities[2];
    pthread_attr_setschedparam(&attr3, &myparam);
}

printf("Starting execution...\n");
if (using_realtime) {
    printf("Using SCHED_FIFO real-time scheduling\n");
} else {
    printf("Using SCHED_OTHER (run with sudo for real-time scheduling)\n");
}
printf("=====\n");

// Set start time then immediately start tasks
clock_gettime(CLOCK_MONOTONIC, &program_start_time);

ret = pthread_create(&thread1, &attr1, task_function, &task1_id);
if (ret != 0) {
    printf("ERROR: pthread_create failed for task 1: %s\n", strerror(ret));
    return 1;
}

ret = pthread_create(&thread2, &attr2, task_function, &task2_id);
if (ret != 0) {
    printf("ERROR: pthread_create failed for task 2: %s\n", strerror(ret));
    return 1;
}

ret = pthread_create(&thread3, &attr3, task_function, &task3_id);
if (ret != 0) {
    printf("ERROR: pthread_create failed for task 3: %s\n", strerror(ret));
    return 1;
}

// Wait for all tasks to complete
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);
pthread_join(thread3, NULL);

pthread_attr_destroy(&attr1);
pthread_attr_destroy(&attr2);
pthread_attr_destroy(&attr3);

// Summary
printf("\n=====\n");
printf("EXECUTION SUMMARY\n");
printf("=====\n");
printf("Task 1 cycles: %d\n", task1_cycles);
printf("Task 2 cycles: %d\n", task2_cycles);
printf("Task 3 cycles: %d\n", task3_cycles);

if (!deadline_miss_detected)
    printf("\n✓ All deadlines were met.\n");
else
    printf("\n⚠ Deadline misses occurred.\n");

```



```

printf("Task 1 deadline misses: %d\n", deadline_misses_task1);
printf("Task 2 deadline misses: %d\n", deadline_misses_task2);
printf("Task 3 deadline misses: %d\n", deadline_misses_task3);

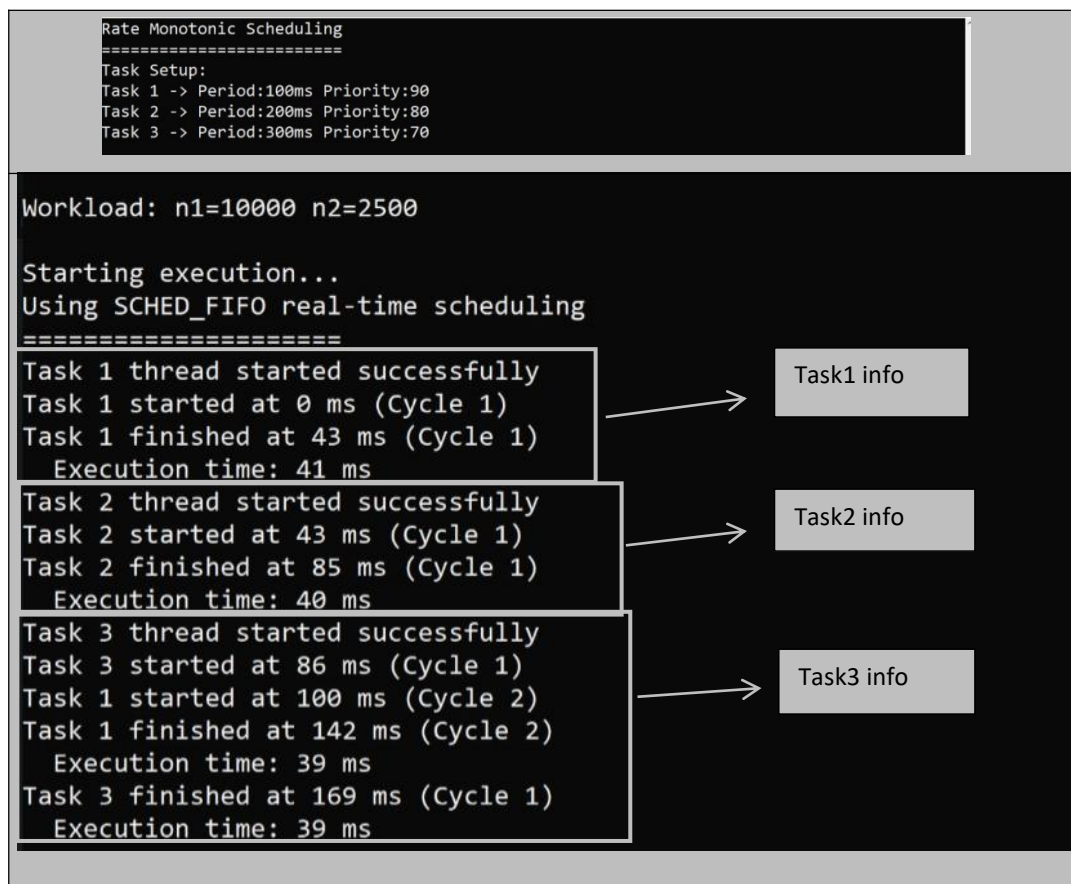
return 0;
}

```

III. Results:

1. Rate monotonic (RM) scheduler (least period -highest priority)

1.1. Tasks information



1.2. Deadline satisfaction

```

=====
EXECUTION SUMMARY
=====
Task 1 cycles: 500
Task 2 cycles: 500
Task 3 cycles: 500

☐ All deadlines were met.
Task 1 deadline misses: 0
Task 2 deadline misses: 0
Task 3 deadline misses: 0

```

1.3. Comment

- The Rate Monotonic (RM) scheduler operated correctly under the given workload. All three periodic tasks executed their full 500 cycles without any deadline misses. Task 1, Task 2, and Task 3 consistently showed execution times in the range of **40–46 ms**, which remained well within their respective periods of 100 ms, 200 ms, and 300 ms. The system followed the RM priority order, assigning the highest priority to the shortest-period task. The stable execution times and absence of deadline violations (deadline misses) confirm that the task set is fully schedulable and that the real-time scheduling using SCHED_FIFO functioned correctly.

2. Non RM scheduling

```

Task Setup:
Task 1 -> Period:100ms Priority:80
Task 2 -> Period:200ms Priority:80
Task 3 -> Period:300ms Priority:80

```

```

=====
EXECUTION SUMMARY
=====
Task 1 cycles: 50
Task 2 cycles: 50
Task 3 cycles: 50

☐ Deadline misses occurred.
Task 1 deadline misses: 8
Task 2 deadline misses: 0
Task 3 deadline misses: 0

```

2.1. Comment

- In this test, I intentionally assigned **the same priority** to all three tasks, even though their periods are different. This means the system was **not actually using Rate Monotonic (RM) scheduling**, because RM requires priorities to be based on task periods (shorter period → higher priority).

Since all tasks had equal priority, the scheduler could not guarantee that the fastest task (Task 1, with a 100 ms period) would run on time. As a result, Task 1 was sometimes delayed by the other tasks and **missed several deadlines**, while Tasks 2 and 3 did not.

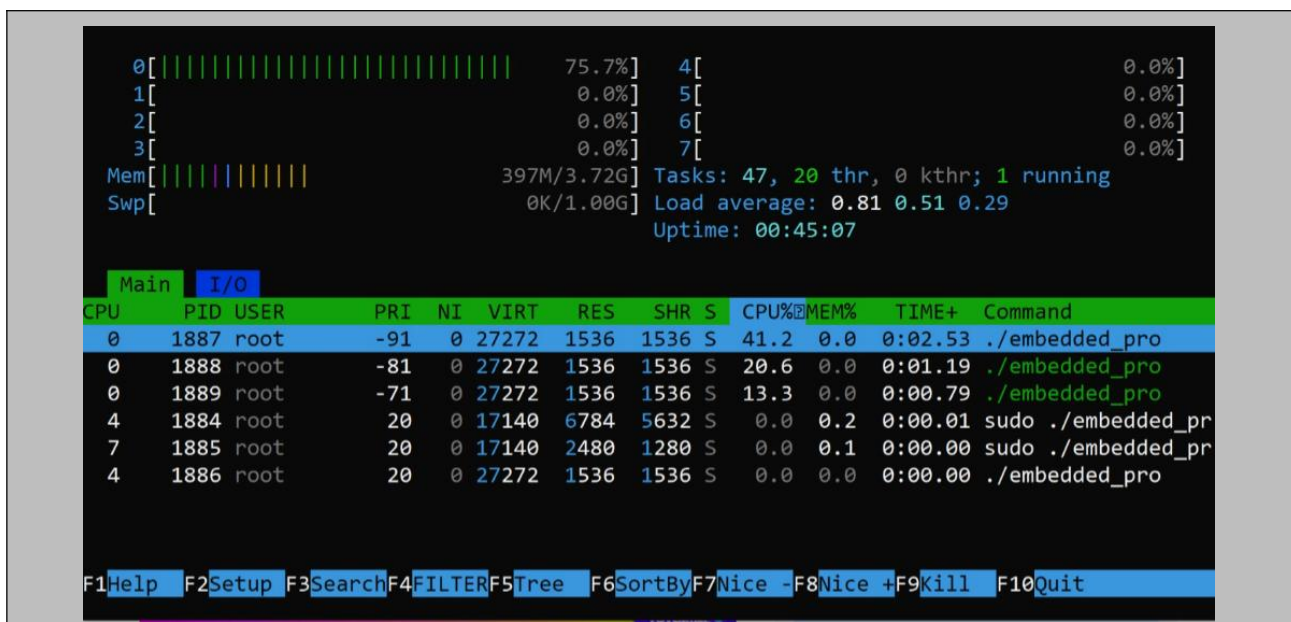
-The results show this clearly:

-Task 1 missed 8 deadlines

-Tasks 2 and 3 missed 0

-This demonstrates that using equal priorities breaks RM assumptions and leads to deadline violations,

3. Single-core CPU utilization of the tasks



3.1. Comment

- Only the three periodic tasks are using the CPU.
- Task 1 has the highest CPU usage (~41%) because it runs most frequently.
- Task 2 and Task 3 use less CPU (~20% and ~13%) based on their lower frequencies.
- With no background applications, CPU time is shared cleanly among the tasks.
- Total utilization stays below 100% (~70%), so the system is not overloaded and runs smoothly.

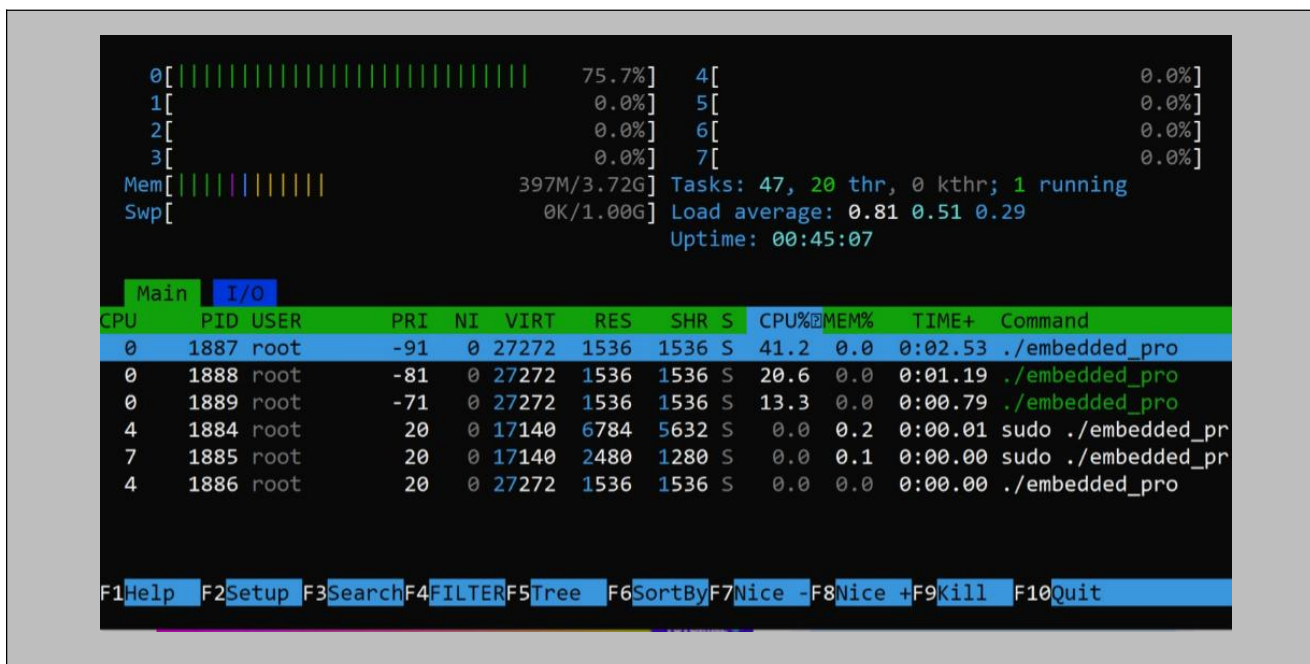
4. Single-core of the tasks without background applications

4.1. Deadline misses

```
=====
EXECUTION SUMMARY
=====
Task 1 cycles: 500
Task 2 cycles: 500
Task 3 cycles: 500

☐ All deadlines were met.
Task 1 deadline misses: 0
Task 2 deadline misses: 0
Task 3 deadline misses: 0
```

4.2. CPU utilization



4.3. Comment

4.3.1. Deadline misses

In the single-core scenario without any background applications running, **all task deadlines were successfully met**.

Each task (Task 1, Task 2, and Task 3) completed its 500 cycles with **0 deadline misses**.

This indicates that the processor had sufficient available time and resources to execute the tasks efficiently without interruption or competition for CPU time.

4.3.2. CPU utilization

The CPU utilization remained **moderate and stable**. The processor handled only the scheduled tasks,

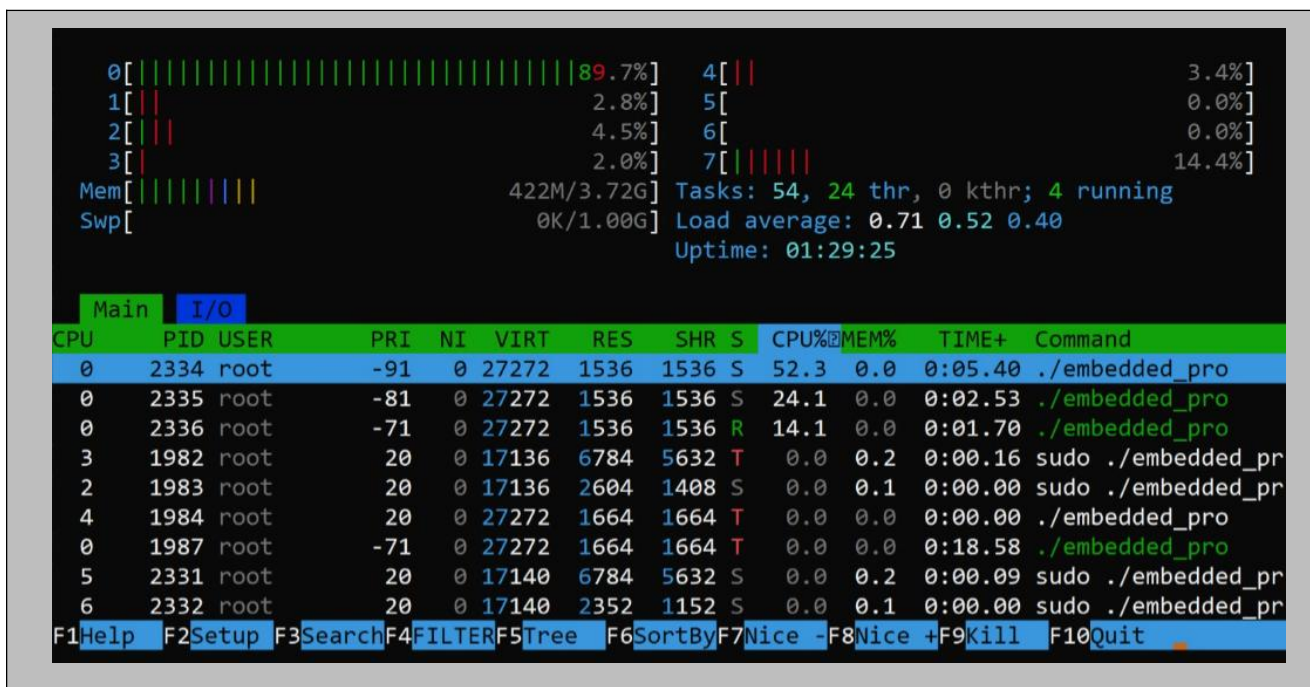
5. Single-core of the tasks with background applications

5.1. Deadline misses

```
=====
EXECUTION SUMMARY
=====
Task 1 cycles: 500
Task 2 cycles: 500
Task 3 cycles: 500

☐ Deadline misses occurred.
Task 1 deadline misses: 26
Task 2 deadline misses: 63
Task 3 deadline misses: 83
```

5.2. CPU utilization



5.3. Comment

5.3.1. Deadline misses

With background applications running, **deadline misses occurred for all tasks:**

- Task 1: 26 misses

- Task 2: 63 misses

- Task 3: 83 misses

This increase in missed deadlines clearly shows that background applications consumed CPU time, reducing the time available for the main tasks. As a result, tasks were **delayed and not able to finish before their deadlines**.

5.3.2. CPU utilization

The CPU usage was **significantly higher** and more irregular. The presence of background processes caused

6. Comparison between parallel loads and only tasks running on single CPU

6.1.1. Deadline misses

In the absence of background applications, all tasks met their deadlines with zero misses. However, when background processes were running, the available CPU time was reduced, leading to several deadline misses across all tasks. This demonstrates that background applications negatively impact the system's ability to meet real-time task deadlines on a single-core processor.

6.1.2. CPU utilization

Without background applications, the single-core CPU showed low to moderate and stable utilization because it was only handling the main tasks. When background applications were introduced, CPU utilization increased significantly due to resource competition and more frequent context switching, which placed a heavier load on the processor and reduced its efficiency

7. Multi-core of the tasks without background applications

7.1. Deadline misses

```
=====
EXECUTION SUMMARY
=====
Task 1 cycles: 500
Task 2 cycles: 500
Task 3 cycles: 500

✓ All deadlines were met.
Task 1 deadline misses: 0
Task 2 deadline misses: 0
Task 3 deadline misses: 0
```

7.2. CPU utilization

```
op - 07:53:25 up 14:31, 1 user, load average: 0.96, 1.10, 1.23
Threads: 264 total, 2 running, 262 sleeping, 0 stopped, 0 zombie
Cpu(s): 30.4 us, 0.8 sy, 0.0 ni, 68.7 id, 0.0 wa, 0.0 hi, 0.1 si, 0.0 st
Mem : 2016.4 total, 525.3 free, 760.1 used, 731.0 buff/cache
Mem Swap: 8683.0 total, 8683.0 free, 0.0 used, 1001.4 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND	P
3336	root	-81	0	27112	528	472	S	49.2	0.0	0:03.21	rm_roaa	1
3337	root	-71	0	27112	528	472	S	24.6	0.0	0:01.61	rm_roaa	0
3338	root	-61	0	27112	528	472	S	16.3	0.0	0:01.06	rm_roaa	2
622	root	20	0	157092	51152	31088	S	1.7	2.5	2:38.70	Xorg	2
1228	pi	20	0	92428	33376	24560	R	1.7	1.6	0:49.21	x-termi+	2
1177	pi	20	0	103716	43540	27504	S	0.3	2.1	0:13.40	geany	2
1366	pi	20	0	10784	3712	3080	S	0.3	0.2	0:36.44	top	2
1	root	20	0	16960	8872	7184	S	0.0	0.4	0:01.83	systemd	2
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd	1
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp	0
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_par+	0
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker+	0
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	mm_perc+	1
9	root	20	0	0	0	0	S	0.0	0.0	0:00.05	ksofttir+	0
10	root	20	0	0	0	0	I	0.0	0.0	0:03.52	rcu_sch+	2
11	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_bh	0
12	root	rt	0	0	0	0	S	0.0	0.0	0:00.16	migrati+	0

7.3. Comment

7.3.1. Deadline misses

All three real-time tasks completed their 500 cycles with **zero deadline misses**. The CPU load was light, and the scheduler efficiently distributed the tasks across multiple cores. Even at low load, **task migration** helped balance execution, ensuring each task received enough CPU time to meet its deadlines.

7.3.2. CPU utilization

CPU usage is relatively low, with the real-time tasks spread across cores and plenty of idle time available.

8. Multi-core of the tasks with background applications

8.1. Deadline misses

```
=====
EXECUTION SUMMARY
=====
Task 1 cycles: 500
Task 2 cycles: 500
Task 3 cycles: 500

✓ All deadlines were met.
Task 1 deadline misses: 0
Task 2 deadline misses: 0
Task 3 deadline misses: 0
```

8.2. CPU utilization

```
op - 07:57:40 up 14:36, 1 user, load average: 1.63, 1.04, 1.15
threads: 281 total, 2 running, 279 sleeping, 0 stopped, 0 zombie
Cpu(s): 49.9 us, 1.5 sy, 0.0 ni, 48.5 id, 0.0 wa, 0.0 hi, 0.1 si, 0.0 st
iB Mem : 2016.4 total, 407.7 free, 870.4 used, 738.3 buff/cache
iB Swap: 8683.0 total, 8683.0 free, 0.0 used. 882.5 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND	P
3390	root	-81	0	27112	524	468	S	49.2	0.0	0:03.47	rm_roaa	0
3391	root	-71	0	27112	524	468	S	24.9	0.0	0:01.73	rm_roaa	2
3392	root	-61	0	27112	524	468	S	16.3	0.0	0:01.18	rm_roaa	1
3369	pi	20	0	358928	43392	22344	S	11.6	2.1	0:05.84	pgzrun	2
3375	pi	20	0	365312	49824	22408	S	11.6	2.4	0:05.04	pgzrun	2
3356	pi	20	0	358440	42760	22324	S	11.3	2.1	0:06.72	pgzrun	2
3363	pi	20	0	359520	44032	22492	S	10.0	2.1	0:05.10	pgzrun	2
3349	pi	20	0	360984	45172	22356	S	8.3	2.2	0:04.99	pgzrun	2
622	root	20	0	164856	57924	37596	S	3.3	2.8	2:42.11	Xorg	2
1076	pi	-6	0	1772048	29564	20044	S	3.3	1.4	2:59.95	alsa-si+	2
1228	pi	20	0	92428	33376	24560	S	2.3	1.6	0:51.05	x-termi+	2
3355	pi	20	0	360984	45172	22356	S	1.0	2.2	0:00.31	pgzrun	2
1049	pi	20	0	1772048	29564	20044	S	0.7	1.4	0:56.39	pulseau+	2
3362	pi	20	0	358440	42760	22324	S	0.7	2.1	0:00.32	pgzrun	2
3368	pi	20	0	359520	44032	22492	S	0.7	2.1	0:00.25	pgzrun	2
3378	pi	20	0	365312	49824	22408	S	0.7	2.4	0:00.08	pgzrun	2
3380	pi	20	0	365312	49824	22408	S	0.7	2.4	0:00.25	pgzrun	2

8.3. Comment

8.3.1. Deadline misses

Even with background applications increasing overall CPU usage, the real-time tasks still experienced zero deadline misses. The system handled the extra load by actively migrating the tasks to less busy cores, preventing interference and keeping their execution on time. Task migration was the main factor that maintained deadline guarantees under higher system load.

8.3.2. CPU utilization

Although some background applications are running, the **CPU utilization remains almost the same** as before, with only a slight increase. Because the system is still far from fully loaded, the scheduler can easily **migrate the real-time tasks between cores**, preventing interference and ensuring **zero deadline misses**.

9. Comparison between parallel loads and only tasks running on multi core

9.1.1. Deadline misses

In both cases—before and after running background applications—the three real-time tasks completed all 500 cycles with zero deadline misses. The key reason deadlines are still met is task migration across CPU cores, which allows the scheduler to move the tasks to less busy cores whenever needed. As a result, the increased background activity does not interfere with real-time performance.

9.1.2. CPU utilization

The CPU utilization in both cases is **very similar**, with only a slight increase when background applications are running. The system remains far from fully loaded in both scenarios, meaning there is plenty of available processing capacity. Because the load difference is small, the background applications do not significantly affect how much CPU the real-time tasks can use.

IV. Full code

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sched.h>
#include <time.h>
#include <math.h>
#include <errno.h>
#include <string.h>

// ===== Configuration =====

// Task periods in milliseconds
const long periods_ms[3] = {100, 200, 300};

// RMS priorities (shorter period = higher priority)
int rms_priorities[3] = {90, 80, 70};

// Global flags and counters
static int deadline_miss_detected = 0;
static int using_realtime = 0;

// Workload loop sizes (simulated CPU load)
const long n1 = 10000;
const long n2 = 2500;

// Timestamp marking program start
static struct timespec program_start_time;

// Cycle counters
static int task1_cycles = 0;
static int task2_cycles = 0;
static int task3_cycles = 0;

// Deadline miss counters
static int deadline_misses_task1 = 0;
static int deadline_misses_task2 = 0;
static int deadline_misses_task3 = 0;

// ===== Time Helpers =====

// Get CPU time for the current thread (excludes time when preempted)
long get_thread_cpu_time_us() {
    struct timespec ts;
    clock_gettime(CLOCK_THREAD_CPUTIME_ID, &ts);
    return ts.tv_sec * 1000000 + ts.tv_nsec / 1000;
}

// Add microseconds to a timespec structure
void timespec_add_us(struct timespec *t, long us) {
    t->tv_sec += us / 1000000;
    t->tv_nsec += (us % 1000000) * 1000;
    if (t->tv_nsec >= 1000000000) {
        t->tv_sec++;
        t->tv_nsec -= 1000000000;
    }
}
```

```

// Compare two timespec values
int timespec_cmp(struct timespec *a, struct timespec *b) {
    if (a->tv_sec != b->tv_sec)
        return (a->tv_sec > b->tv_sec) ? 1 : -1;
    if (a->tv_nsec != b->tv_nsec)
        return (a->tv_nsec > b->tv_nsec) ? 1 : -1;
    return 0;
}

// Milliseconds since program start (wall clock time)
long get_elapsed_ms() {
    struct timespec current_time;
    clock_gettime(CLOCK_MONOTONIC, &current_time);

    long sec = current_time.tv_sec - program_start_time.tv_sec;
    long nsec = current_time.tv_nsec - program_start_time.tv_nsec;

    if (nsec < 0) {
        sec--;
        nsec += 1000000000;
    }
    return sec * 1000 + nsec / 1000000;
}

// Current wall clock time in microseconds
long get_time_us() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000000 + ts.tv_nsec / 1000;
}

// ===== Workload Simulation =====

// Simple nested loops used to generate CPU load
void simulate_workload() {
    volatile int a;
    for (long i = 0; i < n1; i++) {
        for (long j = 0; j < n2; j++) {
            a = j / 2;
        }
    }
    (void)a;
}

// ===== Task Function =====

// Periodic task executed by each thread
void *task_function(void *arg) {
    int task_id = *(int *)arg;
    struct timespec next_wakeup = program_start_time;

    printf("Task %d thread started successfully\n", task_id);

    for (int k = 0; k < 500; k++) {

        // Update cycle count
        if (task_id == 1) task1_cycles++;
        if (task_id == 2) task2_cycles++;
        if (task_id == 3) task3_cycles++;

        // Print start timestamp (wall clock)
        long start_wall_ms = get_elapsed_ms();
        printf("Task %d started at %ld ms (Cycle %d)\n",

```

```

// Measure ACTUAL CPU execution time (excludes preemption time)
long cpu_start_us = get_thread_cpu_time_us();
long wall_start_us = get_time_us();

simulate_workload();

long cpu_end_us = get_thread_cpu_time_us();
long wall_end_us = get_time_us();

long cpu_time_us = cpu_end_us - cpu_start_us;
long wall_time_us = wall_end_us - wall_start_us;

// Print end timestamp (wall clock)
long finish_wall_ms = get_elapsed_ms();
printf("Task %d finished at %ld ms (Cycle %d)\n",
       task_id, finish_wall_ms, k + 1);

// Execution duration in ms (actual CPU time only)
printf(" Execution time: %.0f ms\n", cpu_time_us / 1000.0);

// Compute next activation time
next_wakeup.tv_sec += periods_ms[task_id - 1] / 1000;
next_wakeup.tv_nsec += (periods_ms[task_id - 1] % 1000) * 1000000;
if (next_wakeup.tv_nsec >= 1000000000) {
    next_wakeup.tv_sec++;
    next_wakeup.tv_nsec -= 1000000000;
}

// Check for deadline miss
struct timespec now;
clock_gettime(CLOCK_MONOTONIC, &now);

if (timespec_cmp(&now, &next_wakeup) > 0) {
    printf("DEADLINE MISS: Task %d (Cycle %d)\n", task_id, k + 1);
    deadline_miss_detected = 1;

    if (task_id == 1) deadline_misses_task1++;
    if (task_id == 2) deadline_misses_task2++;
    if (task_id == 3) deadline_misses_task3++;
}

// Sleep until next period
clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next_wakeup, NULL);
}

printf("Task %d completed all cycles\n", task_id);
return NULL;
}

```

```
// ===== Main Function =====

int main() {
    printf("Rate Monotonic Scheduling\n");
    printf("=====\n");

    int task1_id = 1, task2_id = 2, task3_id = 3;
    pthread_t thread1, thread2, thread3;
    pthread_attr_t attr1, attr2, attr3;
    cpu_set_t cpu1, cpu2, cpu3;
    struct sched_param myparam;
    int ret;

    printf("Task Setup:\n");
    printf("Task 1 -> Period:100ms Priority:%d\n", rms_priorities[0]);
    printf("Task 2 -> Period:200ms Priority:%d\n", rms_priorities[1]);
    printf("Task 3 -> Period:300ms Priority:%d\n", rms_priorities[2]);
    printf("Workload: n1=%ld n2=%ld\n", n1, n2);

    // ----- Task 1 (highest priority) -----
    pthread_attr_init(&attr1);
    pthread_attr_setinheritsched(&attr1, PTHREAD_EXPLICIT_SCHED);

    ret = pthread_attr_setschedpolicy(&attr1, SCHED_FIFO);
    if (ret != 0) {
        printf("Warning: Cannot set SCHED_FIFO (need root). Using default scheduling.\n");
        pthread_attr_setschedpolicy(&attr1, SCHED_OTHER);
    } else {
        using_realtime = 1;
    }

    CPU_ZERO(&cpu1);
    CPU_SET(0, &cpu1);
    pthread_attr_setaffinity_np(&attr1, sizeof(cpu_set_t), &cpu1);

    if (using_realtime) {
        myparam.sched_priority = rms_priorities[0];
        pthread_attr_setschedparam(&attr1, &myparam);
    }

    // ----- Task 2 -----
    pthread_attr_init(&attr2);
    pthread_attr_setinheritsched(&attr2, PTHREAD_EXPLICIT_SCHED);
    pthread_attr_setschedpolicy(&attr2, using_realtime ? SCHED_FIFO : SCHED_OTHER);

    CPU_ZERO(&cpu2);
    CPU_SET(0, &cpu2);
    pthread_attr_setaffinity_np(&attr2, sizeof(cpu_set_t), &cpu2);

    if (using_realtime) {
        myparam.sched_priority = rms_priorities[1];
        pthread_attr_setschedparam(&attr2, &myparam);
    }

    // ----- Task 3 -----
    pthread_attr_init(&attr3);
    pthread_attr_setinheritsched(&attr3, PTHREAD_EXPLICIT_SCHED);
    pthread_attr_setschedpolicy(&attr3, using_realtime ? SCHED_FIFO : SCHED_OTHER);

    CPU_ZERO(&cpu3);
    CPU_SET(0, &cpu3);
    pthread_attr_setaffinity_np(&attr3, sizeof(cpu_set_t), &cpu3);
}
```

```

if (using_realtime) {
    myparam.sched_priority = rms_priorities[2];
    pthread_attr_setschedparam(&attr3, &myparam);
}

printf("Starting execution...\n");
if (using_realtime) {
    printf("Using SCHED_FIFO real-time scheduling\n");
} else {
    printf("Using SCHED_OTHER (run with sudo for real-time scheduling)\n");
}
printf("=====\n");

// Set start time then immediately start tasks
clock_gettime(CLOCK_MONOTONIC, &program_start_time);

ret = pthread_create(&thread1, &attr1, task_function, &task1_id);
if (ret != 0) {
    printf("ERROR: pthread_create failed for task 1: %s\n", strerror(ret));
    return 1;
}

ret = pthread_create(&thread2, &attr2, task_function, &task2_id);
if (ret != 0) {
    printf("ERROR: pthread_create failed for task 2: %s\n", strerror(ret));
    return 1;
}

ret = pthread_create(&thread3, &attr3, task_function, &task3_id);
if (ret != 0) {
    printf("ERROR: pthread_create failed for task 3: %s\n", strerror(ret));
    return 1;
}

// Wait for all tasks to complete
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);
pthread_join(thread3, NULL);

pthread_attr_destroy(&attr1);
pthread_attr_destroy(&attr2);
pthread_attr_destroy(&attr3);

// Summary
printf("\n=====\n");
printf("EXECUTION SUMMARY\n");
printf("=====\n");
printf("Task 1 cycles: %d\n", task1_cycles);
printf("Task 2 cycles: %d\n", task2_cycles);
printf("Task 3 cycles: %d\n", task3_cycles);

if (!deadline_miss_detected)
    printf("\n✓ All deadlines were met.\n");
else
    printf("\n⚠ Deadline misses occurred.\n");

printf("Task 1 deadline misses: %d\n", deadline_misses_task1);
printf("Task 2 deadline misses: %d\n", deadline_misses_task2);
printf("Task 3 deadline misses: %d\n", deadline_misses_task3);

return 0;
}

```